

Research Project: Experts Search Engine

Izabela Ewa Hetmanowska *izhe@itu.dk*

Miguel Angel Castro Gimenez *migca@itu.dk*

Supervisor: Toine Bogers *tobo@itu.dk*

2026-05-15

Contents

1	Introduction	4
1.1	Repository	4
2	Related work	5
2.1	Wiki ITU	5
2.2	Pure	5
2.3	Other Projects at ITU	6
3	Methodology	6
3.1	Data collection	6
3.1.1	Pure	6
3.1.2	Mapping Pure data to CSV	7
3.1.3	Danish Research Portal	10
3.1.4	LearnIT	11
3.2	Data cleaning and loading	12
3.2.1	Pure	14
3.2.2	Danish Research Portal	15
3.2.3	LearnIT	18
4	Architecture	19
4.1	Database design	20
4.2	Normalisation and Database creation	23
4.2.1	Normalization and indexing	23
4.2.2	Third Normal Form (3NF)	23
4.2.3	Schema definition	23
4.3	Search Engine - Back end	25
4.3.1	System Overview	25
4.3.2	indexed_experts - flattened table	26
4.3.3	Data Normalization and Preprocessing	27
4.3.4	Query Classification	27
4.3.5	Multi-Field Retrieval Using BM25F	28
4.3.6	Candidate Retrieval and Database Integration	29
4.3.7	Score Adjustment and Re-ranking	29
4.3.8	Score Normalization	30
4.3.9	REST API with Fast-API	30
4.4	Web application - Front End	31
5	Discussion and Conclusion	34
5.1	Limitations	34
5.1.1	Data collection	34
5.1.2	Search engine	35
5.2	Improvements	36
5.2.1	Search Engine - Back end	36
5.2.2	Data collection	36
5.2.3	Front-end	37
5.2.4	Deployment	37
5.3	Conclusion	37

1 Introduction

Identifying a suitable supervisor or expert for a research project or thesis is often a complex and time-consuming process. Students typically begin by defining a topic of interest, after which they must navigate departments, research groups, projects, and academic organizations in search of individuals with relevant expertise. This process frequently involves contacting multiple academics individually to determine not only whether they possess the necessary knowledge, but also whether they are available or willing to provide guidance. Institutional affiliation alone is often insufficient as an indicator of expertise, since academics associated with a department or organization may not actively work within the specific area of interest. Similar challenges are also faced by public institutions and authorities seeking expert knowledge on matters of societal or scientific importance.

This project focuses specifically on expert discovery within the IT University of Copenhagen (ITU). Although existing academic platforms provide mechanisms for finding researchers and publications, several limitations reduce their effectiveness for expert search and recommendation. One key issue is the lack of transparency in the way search and ranking algorithms operate. Users often cannot understand why certain academics appear higher in search results than others, making it difficult to assess the reliability or relevance of the recommendations presented.

Another challenge concerns the completeness and integration of academic data. Existing systems used at ITU may not contain publications produced by researchers prior to their affiliation with the university, resulting in incomplete representations of academic expertise. Furthermore, information from different areas of academic activity is fragmented across separate platforms. For example, the Pure publication management system stores research output, but does not integrate information about courses taught, supervision experience, or other teaching-related activities. As a result, users are provided with only a partial view of an academic's qualifications and areas of expertise.

In addition, existing systems such as Pure or the Danish Research Portal are designed to support a broad range of administrative and publication-related functions. While these systems are valuable for research management and dissemination, their complexity and extensive metadata can introduce unnecessary noise for users whose primary objective is simply to identify an appropriate expert in a given field.

The main objective of this research project has therefore been to design and develop a more transparent and user-focused expert search engine for ITU academics. To achieve this, we collected data from multiple academic sources, cleaned and standardized the information, and integrated it into a unified database. This approach enables richer and more accurate expert profiles while also providing greater context regarding an academic's research interests, publications, and teaching activities. Furthermore, we developed a Minimum Viable Product (MVP) in the form of a web-based search engine that demonstrates the practical application of our approach. The system emphasizes transparency by clearly exposing the implemented search and ranking mechanisms while presenting concise and relevant information about each expert in a manner that avoids overwhelming the user.

1.1 Repository

The below GitHub repository contains all scripts used to extract, clean and map the data from LearnIt, the Danish Research Portal and Pure in the folder `./Data_collection`.

Moreover, both `server` and `client` have been containerized for Docker and can be found in folder `./App`. Commands to run both `server` and `client` can be found in the `README.md` file in the root directory.

Expert Search Engine

To clone the repository, open a terminal in the directory where the project should be stored and execute the following command:

```
git clone https://github.com/izahetmanowska/ExpertSearch.git
```

2 Related work

2.1 Wiki ITU

The ITU Computer Science wiki hosts a MediaWiki page titled *Supervisors for projects and theses*¹, which functions as a lightweight expert-discovery portal for students. The page is organised as a directory-style listing of supervisors with a table of contents and deep links to individual entries, enabling rapid navigation across a large set of potential supervisors.

Each supervisor is presented in a relatively consistent “profile snippet” format, typically including (i) free-text research interests and topic keywords, (ii) contact information (most commonly an email link), and (iii) external resources such as personal homepages, lab pages, and project-idea pages (and occasionally social media links). The portal is explicitly oriented towards thesis and project supervision: many entries include concrete project ideas, and several also provide availability-related constraints (e.g., limited supervision capacity, temporary absence, or notes regarding part-time lecturer permissions).

In addition to the supervisor list itself, the page embeds student-facing guidance and links to administrative resources (e.g., pages describing thesis requirements and intended learning outcomes), which can support expectation-setting during supervisor selection. The wiki also includes a global site search; however, this operates at the level of the wiki as a whole rather than as a dedicated expert-search interface.

The directory offers broad coverage but relies on manual browsing and self-maintained profiles, which may result in outdated or missing information.

2.2 Pure

Pure² is a research information management platform used at the IT University of Copenhagen and across other Danish universities. It supports search over multiple entity types—including organisations, researchers, projects, publications, datasets, media, activities, and prizes—and is therefore not an expert-search tool in isolation, but a broader system for registering and disseminating research output. In practice, this means that a user searching for a term must first navigate a mixed set of results and then filter specifically to the researcher category in order to obtain an expert-focused view.

From a search perspective, Pure provides two complementary query modes: a *concept* search that targets key concepts extracted from titles and abstracts, and a *text* search

¹Supervisors for projects and theses

²Pure ITU

over indexed content such as names, titles, and descriptions. It also offers an advanced search interface. Researcher profiles contain rich information, including name, title, ORCID, email, and occasionally office location, alongside self-authored summaries and user-provided keywords. Profiles also link to associated publications and projects, and include additional exploratory features such as suggested similar profiles, collaboration networks, and a “fingerprint” of keyphrases/keywords with associated scores.

While Pure exposes a broad and feature-rich interface, it provides limited transparency about ranking and scoring. The ordering of researcher results and the rationale behind profile features such as “similar profiles” and fingerprint scores are not clearly explained, which makes it difficult for users to understand why certain researchers are surfaced for a given query. Finally, both the front end and the API reflect the platform’s general-purpose scope: the large number of entities, endpoints, and fields can introduce unnecessary complexity for users and developers who require a focused expert-search experience.

2.3 Other Projects at ITU

A closely related line of work is the design and evaluation of a web-based platform for matching students with thesis supervisors at ITU [1]. The platform supports two search modalities: free-text input and keyword-based selection, with the goal of helping students discover suitable supervisors based on research interests. A study by another group of ITU students [2] emphasizes that, beyond retrieval accuracy, such systems must provide transparency about why specific supervisors are recommended, as users’ trust and perceived fairness depend on understandable reasoning.

The platform [1] was evaluated with a mixed-methods approach combining task-based user testing (including cognitive walkthrough and remote testing) with standardized questionnaires (SUS and UEQ) and qualitative feedback. Quantitative results indicated above-average usability, but qualitative findings revealed two issues that are particularly relevant for expert-search systems: (i) an *explainability gap*, where users can operate the interface but do not understand the rationale behind the ranking/recommendations, and (ii) a *dual-mode integration problem*, where offering both free-text and keyword search can reduce clarity if the two modes are not coherently connected [2]. The proposed improvements on the study [2] such as clearer recommendation explanations (e.g., highlighting matched terms and showing supporting items), better keyword organization, and iterative query refinement from the results view map directly onto the requirements of an expert search engine that aims to be both effective and transparent.

3 Methodology

3.1 Data collection

3.1.1 Pure

The Pure platform provides an API for accessing institutional research data. For this project, an access token was obtained and used to retrieve data related to Persons, Research Outputs, and Projects.

Data concerning Persons and Projects was extracted manually through individual calls to the corresponding Pure API endpoints, after which the responses were stored as JSON files for further processing.

In contrast, the volume of Research Output data was substantially larger, comprising more than 8,000 records. To facilitate its extraction, a dedicated fetching script, `fetchResearchOutput.js`, was developed. This script reused the API request configuration defined in the `getResearchOutput.http` test file and automated the retrieval of Research Output entries from Pure.

Although the extraction process may appear relatively simple, this design decision was intentional and aligned with the objectives of the project. The primary focus was placed on obtaining, transforming, and mapping the required data as part of a proof-of-concept implementation, rather than developing a fully automated ETL pipeline. Furthermore, based on discussions with Toine B., tutor for this project, research output data at ITU is typically updated by academics approximately once per year. Consequently, the automation of the extraction workflow was considered a lower priority within the scope of this work.

For this reason, the implementation details of the fetching script are not discussed further. While the script can be reused for future extractions, additional modifications would be necessary to support a fully streamlined workflow capable of loading data directly into the target database.

More significant to the project was the selection of data attributes extracted from each entity type in Pure and the subsequent mapping of this data into CSV files for database ingestion. These design decisions and mappings are described in detail in the following subsection.

3.1.2 Mapping Pure data to CSV

The processes of database design and data extraction from Pure were carried out simultaneously throughout the project. As a consequence, it was not feasible to directly load the transformed data into the database during the extraction phase, since the database schema was still evolving. To address this, the extracted and transformed data was first exported into intermediary CSV files. These files were subsequently uploaded into a staging environment before being inserted into the final database tables.

Mapping of Persons The script `Data_collection/MappingScripts/mapPersonsToCsv.js` was executed in Node.js to transform selected fields from `Data_collection/PureJSONs/persons.JSON` into a flattened CSV structure suitable for relational database import.

The following attributes were extracted and mapped:

- `UUID ← items[*].uuid`
- `firstName ← items[*].name.firstName`
- `lastName ← items[*].name.lastName`
- `email ← items[*].staffOrganizationAssociations[0].emails[0].value`
- `jobTitleId ← integer identifier mapped from items[*].staffOrganizationAssociations[0].jobTitle.term.en_GB`

Additionally, the script generated a lookup table in CSV format (`job_title_lookup.csv`), allowing `jobTitleId` to function as a foreign key within the database schema.

Mapping of Projects Project data was transformed using the script `Data_collection/MappingScripts/mapProjectsToCsv.js`. The script extracted selected fields from `projects.JSON` and produced a flattened CSV representation intended for insertion into the `Project` table.

The mapped fields were as follows:

- `uuid ← items[*].uuid`
- `title ← items[*].title.en_GB`
- `periodStartDate ← items[*].period.startDate`
- `periodEndDate ← items[*].period.endDate`
- `description ← first available items[*].descriptions[*].value.en_GB`
- `portalUrl ← items[*].portalUrl`

Mapping of Research Outputs Research output data was processed using the script `Data_collection/MappingScripts/mapPapersToCsv.js`. The script transformed selected fields from `researchOutput.JSON` into a flat CSV structure for insertion into the `Paper` table.

The following mappings were applied:

- `uuid ← items[*].uuid`
- `year ← items[*].submissionYear`
- `title ← items[*].title.value`
- `subtitle ← items[*].subTitle.value`
- `university ← constant value ITU`
- `type ← items[*].type.term.en_GB`
- `abstract ← items[*].abstract (prioritising en_GB and subsequently da_DK)`
- `file ← items[*].portalUrl`
- `totalNumberOfContributors ← items[*].totalNumberOfContributors`

Mapping of Paper Contributors The script `Data_collection/MappingScripts/mapPaperContributorsToCsv.ts` processed contributor information associated with research outputs and generated a CSV file representing contributor relationships.

The resulting dataset contained the following fields:

- `paperUUID ← item.uuid`
- `personUUID ← resolved from person.csv through name matching for internal contributors`
- `personName ← contributor.name.firstName`
- `personLastname ← contributor.name.lastName`
- `externalContributor ← boolean evaluation of contributor.typeDiscriminator == 'ExternalContributorAssociation'`
- `role ← contributor.role.term.en_GB`

The `personUUID` matching procedure employed multiple strategies, including exact full-name matching, accent-normalised matching, and initial-plus-surname matching. External contributors and unmatched individuals were assigned null values.

Mapping of Project Participants Participant information associated with projects was processed using the script `Data_collection/MappingScripts/mappingProjectParticipants.ts`.

The generated CSV file included the following mappings:

- `projectUUID ← project.uuid`
- `personName ← participant.name.firstName`
- `personLastName ← participant.name.lastName`
- `role ← participant.role.term.en_GB`
- `externalContributor ← boolean evaluation of participant.typeDiscriminator == 'ExternalParticipantAssociation'`
- `personUUID ← resolved from person.csv through exact first-name and last-name matching`

If a participant was external, ambiguous, or unmatched, the corresponding `personUUID` field was left empty.

Keyword Extraction and Categorisation Keywords extracted from Pure were initially envisioned as a mechanism for categorising and filtering content within the search engine. The intended functionality was to allow users to combine multiple keywords as filters, thereby retrieving entities associated with all selected terms.

However, during the extraction process it became evident that keywords in Pure are stored as free-text entries. As a result, users often entered highly specific or inconsistent terms, limiting the effectiveness of direct keyword-based categorisation. An example of such a keyword was:

17.3% blue notebook experiment (2008)

Terms of this nature were considered too specific to support meaningful aggregation across multiple entities.

To mitigate this issue, keywords were decomposed into individual words. Although this approach does not fully preserve semantic meaning, it enabled partial reuse of extracted information. For example, the previous keyword could be transformed into the individual terms *blue*, *notebook*, and *experiment*, thereby allowing relationships to be established between this item and others that would contain the keyword *experiment*.

The keyword extraction procedure was identical across Research Outputs, Persons, and Projects. The process can be summarised as follows:

1. Locate the keyword array associated with the entity.
2. Extract each keyword entry.
3. Split each keyword into individual terms.
4. Check whether the resulting term already exists in the keyword dataset.
 - (a) If the term does not exist, create a new keyword record.
 - (b) If the term already exists, retrieve its existing identifier.
5. Store the relationship between the keyword identifier and the corresponding entity identifier in the appropriate relational table.

The keyword dataset was designed as a table containing unique keyword identifiers and keyword terms, while relationship tables such as `Paper_keywords` linked entities to their associated keywords.

Although keyword extraction and categorisation were implemented during the data preparation phase, keywords were not integrated into the current implementation of the search engine. This decision was made because the present project focused on delivering a simplified proof-of-concept system. Nevertheless, the extraction and structuring of keyword data were considered foundational steps toward a more advanced search and recommendation system intended for future thesis work.

3.1.3 Danish Research Portal

To enrich the initial dataset, an automated data collection method was implemented using DRP (Danish Research Portal) hosted at <https://forskningsportal.dk>.

Firstly, data on individuals affiliated with the IT University of Copenhagen was accumulated using a Python script that accessed the Pure API through authenticated HTTP

requests. The script retrieved records in batches, extracted first and last names from the JSON responses, and exported the cleaned data to a CSV file.

This dataset was processed using the *pandas* library; basic cleaning procedures were applied, including trimming whitespaces and removing trailing hyphens from last names. A search field was then generated by concatenating the last and first names in the format “*Last name, First name*” to ensure compatibility with DRP searches.

As the DRP platform does not provide a public API for large-scale retrieval, the scraper was developed using the *Playwright* library to automate interactions with the web interface, operating on the preprocessed `persons.csv` file containing standardized person names. Each name was submitted as a query via the portal’s advanced search interface, with the search category restricted to “Contributors” for improved relevance. Following query execution, the script accessed the portal’s export functionality and downloaded the corresponding results as JSON records.

The core workflow of the scraper is illustrated below:

```
choose_contributors(page)
fill_search_value(page, value)
click_search(page)
export_results(page, DOWNLOAD_DIR, safe_filename(value))
```

Downloaded files were stored locally using filenames derived from the search queries. To improve reliability in a dynamic web environment, the scraper incorporated explicit waits, fallback interaction strategies, and exception handling. In cases of error, screenshots were automatically captured to support debugging and validation.

This implementation enabled large-scale extraction of publication data from a platform primarily designed for manual interaction.

3.1.4 LearnIT

The course data was collected from the IT University of Copenhagen’s LearnIT course base hosted at <https://learnit.itu.dk/local/coursebase/> using a web scraper implemented with the *Playwright* library. The scraper automated interactions with a Chromium browser environment, ensuring access to content not available through static requests. The interface was configured to display 50 courses per page; multiple pages were processed sequentially.

For each page, the system identified visible course-title elements through a combination of layout-based heuristics and textual filtering. The method excluded irrelevant interface components while preserving entries matching common title patterns, such as semester indicators or multi-word formulations.

Listing 1: Filtering visible course-title elements

```
for el in candidates:
    try:
        if not el.is_visible():
            continue

        box = el.bounding_box()
        if not box:
            continue

        if box["x"] < 20:
            continue
```

```

if box["y"] < 80:
    continue
if box["width"] < 900:
    continue
if box["height"] < 30 or box["height"] > 120:
    continue

txt = clean_text(el.inner_text())

except Exception:
    continue

if not looks_like_course_title(txt):
    continue

```

This approach was adopted in response to the page structure, which integrated course entries and interface components within the same DOM (Document Object Model) hierarchy.

After a course entry had been identified, the scraper simulated a click event to reveal detailed information. Relevant content was then extracted as a text block and parsed using regular expressions to obtain structured fields, including course name, ECTS credits, semester, language, examination type, teacher(s), and associated programme.

Listing 2: Extraction of structured course information

```

m = re.search(
    rf"{re.escape(label)}\s+(.+) (?=\s+(?:{next_part})\b|$)",
    text,
    re.I,
)

return {
    "ECTS": get("ECTS", ["Semester", "Language", "Exam"]),
    "Teacher": get("Teacher", ["Programme"]),
}

```

The collected results were stored in a `learnit_courses.csv` file with one row per unique course record.

This implementation adopted a hybrid strategy combining browser automation with heuristic text extraction. Rather than relying solely on stable HTML selectors, it uses layout, visibility, and textual patterns to extract data from a dynamic interface. Although adaptable to changing page structures, the scraper remains dependent on assumptions about the website's layout and should therefore be regarded as a heuristic rather than a formally guaranteed approach.

3.2 Data cleaning and loading

As discussed in the previous section, no fully automated ETL pipeline was implemented in this project. Instead, data quality was assessed iteratively at each stage of extraction and transformation. Based on these evaluations, dataset-specific cleaning operations were performed where necessary. While this approach is not optimal for large-scale production systems, it was appropriate for the exploratory and proof-of-concept nature of this project. Future iterations could benefit from the implementation of a fully automated and standardised ETL workflow.

Despite the lack of automation, the overall data cleaning and transformation process remained consistent across all entity types. A uniform sequence of steps was applied to each dataset retrieved from Pure. The following steps describe this process using the `Project` dataset as an example:

1. Staging Tables

For each production table in the database, a corresponding staging table was created. Staging tables were designed to be permissive and unconstrained in order to allow ingestion of raw data without immediate validation failures.

This design choice was necessary due to data quality issues encountered during initial loading attempts. For example, some project records lacked required attributes such as titles, which violated constraints in the production schema and caused insertion failures. Similarly, in the `paper_contributors` dataset, certain entries were missing first or last names, which would also have prevented direct insertion.

Using a staging environment allowed these inconsistencies to be identified and filtered prior to final insertion.

2. Data Loading into Staging

Data was loaded from CSV files into the staging tables using PostgreSQL's `\copy` command:

Listing 3: Loading data to staging table using PL-SQL

```
\copy staging_project FROM '/Users/miguel/Documents/1. ITU/5. Research Project/ExpertSearch/CsvForDB/Project.csv'
WITH (FORMAT csv, HEADER true, NULL '');
```

3. Insertion into Production Tables

After loading and inspecting the staged data, only valid records were inserted into the production tables. Filtering was applied directly within the SQL insertion query to ensure data integrity:

Listing 4: Loading data from staging to production table

```
INSERT INTO project (
    "UUID",
    title,
    period_start_date,
    period_end_date,
    description,
    portal_URL
)
SELECT
    "UUID",
    title,
    period_start_date,
    period_end_date,
    description,
    portal_URL
FROM staging_project
WHERE "UUID" IS NOT NULL
    AND title IS NOT NULL;
```

3.2.1 Pure

The general above-mentioned process was employed for all data extracted from Pure, moreover, a lot of efforts were directed towards extracting value from the data it held regarding keywords.

Keyword Cleaning and Normalisation

The keyword extraction process described in the subsection Data collection resulted in several challenges. Specifically, three main issues were identified:

1. A large number of semantically meaningless keywords.
2. A significant proportion of stopwords and non-informative tokens.
3. Redundant keyword assignments, where multiple variations of similar or identical concepts were associated with a single entity.

For instance, a single person such as “Søren Nielsen” could be associated with keywords such as “applied mathematics” and “discrete mathematics”, which after decomposition resulted in multiple occurrences of the generic term “mathematics”.

To address these issues, a multi-step cleaning approach was applied:

- **Removal of Irrelevant Tokens**

Keywords were initially loaded into a staging table and filtered using SQL queries to remove non-informative or malformed entries. This included both syntactically invalid tokens and overly specific or meaningless strings.

Listing 5: Set of removed keywords

```
('13', 'no', 're', 'ar', 'mt', 'ii', 'bl', 'ot', 'g4', 'co', 'ej', 'ad', 'uc', 'ma', 'px',  
...  
'68w30', 'aaa')
```

- **Stopword Filtering**

A programmatic stopwords list was implemented in the mapping script `mapKeywordsToCsv.ts` to remove high-frequency English function words that do not contribute semantic meaning:

Listing 6: Set of stopwords

```
const STOPWORDS = new Set([  
  'a', 'an', 'and', 'are', 'as', 'at', 'be', 'been', 'being', 'but',  
  'by',  
  ...  
  'with', 'you', 'your', 'yours', 'yourself', 'yourselves'  
]);
```

- **Deduplication Strategy**

Since the objective was to establish binary relationships between keywords and entities rather than to measure frequency, a deduplication step was applied. Only unique pairs of `keyword_id` and entity identifiers (`paper_uuid`, `person_uuid`, `project_uuid`, or `course_uuid`) were retained.

These were then inserted into the corresponding relational tables (`paper_keywords`, `person_keywords`, `project_keywords`, and `course_keywords`), ensuring that each association existed at most once.

3.2.2 Danish Research Portal

1. Data Extraction and Transformation.

A Python-based data transformation pipeline was developed in order to integrate research publication records exported from the Danish Research Portal into a structured dataset. The script processed multiple `.lst` files containing JSON-formatted publication data and combined them into a single CSV file. The overall transformation process is illustrated in figure 1.

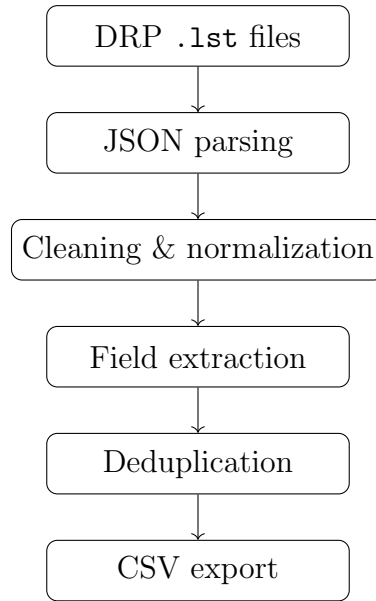


Figure 1: ETL workflow for DRP publication data

To accommodate inconsistencies in the exported data, the script implemented flexible JSON parsing capable of handling multiple objects and irregular structures within individual files.

Listing 7: Flexible parsing of irregular JSON export structures

```
decoder = json.JSONDecoder()
idx = 0
objects = []

while idx < len(content):
    obj, end = decoder.raw_decode(content, idx)

    if isinstance(obj, list):
        objects.extend(obj)
    else:
        objects.append(obj)

    idx = end
```

Preprocessing included whitespace normalization, deduplication, and handling of missing values. Additional information, such as author names, was derived from the file names.

For each publication record, the script extracted key bibliographic information, including the title, publication year, publication type, authors, affiliations, keywords, journal, publisher, DOI, open-access links, and source details. The processed records were exported to a unified UTF-8 encoded CSV file.

Overall, the script functioned as an automated Extract–Transform–Load (ETL) workflow, enabling the systematic organization and standardization of data collected from the DRP scraper.

2. Schema Standardization

A post-processing transformation pipeline was applied to the dataset exported as `danish_research_all.csv` in order to standardize the schema and prepare the data for database integration. The dataset was loaded into a *pandas* DataFrame, and non-essential metadata fields, including source information, publisher data, and affiliations, were removed.

The script then evaluated the completeness of the *doi* [3] and *open_access_url* fields. Missing DOI values were supplemented using values from the *open_access_url* column when available:

```
df["doi"] = df["doi"].fillna(df["open_access_url"])
```

Several columns were subsequently renamed to align with the target database schema:

```
df = df.rename(columns={
    "doi": "file",
    "author_count": "totalNumberOfContributors",
    "lang": "language",
    "id": "researchPortalId",
})
```

Finally, the columns were reordered, unnecessary fields were removed, and the cleaned dataset was exported as `fixed.csv`.

3. Overlap Detection Between PURE and DRP

Publication records from `fixed.csv` were compared against existing entries in `paper.csv` in order to identify records already present in the database. The matching process was based on normalized publication titles derived from existing PURE records.

The script first combined the *title* and *subtitle* fields from `paper.csv`, then normalized both datasets by lowercasing text, removing punctuation, and standardizing whitespace. The titles from both files were then compared:

```
paper_titles_set = {x for x in papers_review["norm_title"] if x
}

is_match = (
    fixed_review["norm_title"].isin(paper_titles_set)
    & fixed_review["norm_title"].ne("")
)
```

Based on this comparison, the dataset was split into matched and unmatched records. Matched records were saved as `rdp_pure_match.csv`, while unmatched, as `fixed_remaining.csv`.

Finally, the script generated review files to support manual validation. `match_review.csv` stored matched titles side by side, while `ambiguous_matches.csv` flagged cases where one title matches multiple existing records.

The remaining publication records in `fixed_remaining.csv` were further processed to integrate duplicate entries based on publication titles. Empty strings were first converted into missing values, records sharing the same title were then grouped, and incomplete data was merged by retaining the first available non-empty value for each column:

```
def merge_group(group):
    result = {}
    for col in group.columns:
        non_null = group[col].dropna()
        result[col] = non_null.iloc[0] if not non_null.empty else [pd.
            NA](http://pd.na/)
    return pd.Series(result)
```

The final dataset was exported as `DRP.csv`. Overall, this step functioned as a quality-control stage, ensuring that duplicate publication records were excluded during database integration.

4. Relational Data Structuring

The cleaned DRP dataset was further transformed into database-ready relational tables. First, selected publication fields were extracted from `DRP.csv` and renamed to match the target database schema for `DRPPaper.csv`.

```
df_new = df[[
    "researchPortalId", "year", "title", "type",
    "abstract", "file", "totalNumberOfContributors"
]].rename(columns={
    "researchPortalId": "DRPId"
})
```

Contributor information was then extracted from both the *author* and *authors* columns. Names were split, cleaned, deduplicated, and stored as unique person records in `DRPPerson.csv`.

```
if author_1:
    combined.extend(author_1.split(";"))
if author_2:
    combined.extend(author_2.split(";"))

for person in combined:
    person = person.strip()
    if person:
        people.add(person)
```

To align DRP contributors with existing person records, names were normalized and matched against `person.csv`. When a match was found, the generated DRP person identifier was replaced with the existing UUID.

```
merged = drp.merge(
    person[["UUID", "firstName_norm", "lastName_norm"]],
    on=["firstName_norm", "lastName_norm"],
    how="left"
)

merged["DRPPersonId"] = merged["UUID"].combine_first(merged["
    DRPPersonId"])
```

Finally, a many-to-many contributor table was generated by linking each publication to its associated contributors. This resulted in `DRPPaperContributors.csv`, containing publication IDs, contributor IDs, and contributor names.

Although keyword matching was initially also implemented by comparing normalized keywords against publication titles and abstracts, this output was not used in the final database integration.

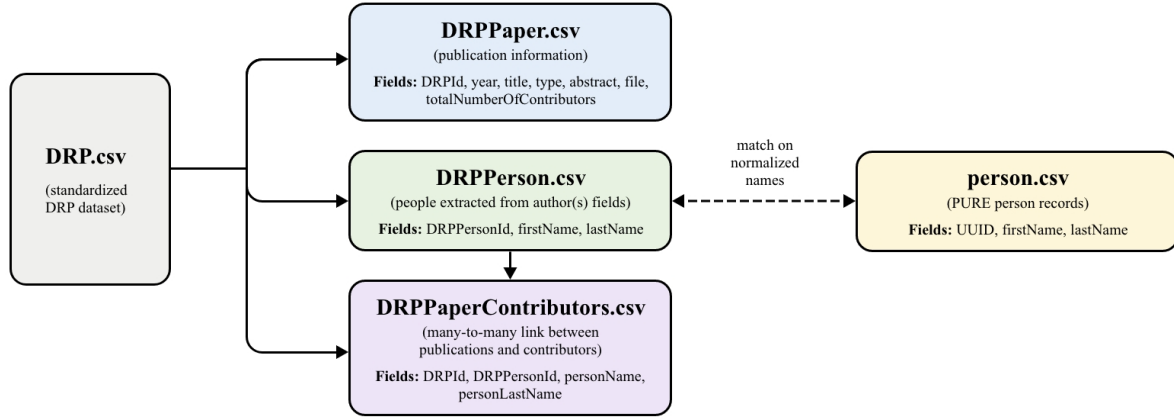


Figure 2: Relational structuring workflow for DRP publication data

3.2.3 LearnIT

The collected dataset was cleaned and standardized in preparation for database integration and relational mapping. The process consisted of two stages. First, a cleaned and standardized `Course.csv` file was generated from the original dataset. Second, the cleaned course data was combined with information from `person.csv` (PURE) to construct `PersonTeachesCourse.csv`, linking courses to their corresponding teachers.

The workflow began by loading `learnit_courses.csv` into a *pandas* DataFrame. Course-period information was extracted from course titles when available; otherwise, values from the *Semester* field were used. Course titles were standardized by removing semester expressions and normalizing punctuation and whitespace, while the *language* field was restricted to “English” and “Danish”. Duplicate records were removed based on period, title, and language. A sequential id column was introduced before exporting the cleaned dataset to `Course.csv`.

The second stage focused on linking courses with lecturers. To support reliable matching, text normalization procedures such as lowercasing, Unicode normalization, and whitespace standardization were applied across datasets. Lookup dictionaries were then used to map cleaned course titles and periods to course IDs, and normalized teacher names to UUIDs from `person.csv`.

```

course_lookup = dict(zip(
    zip(course_df["title_clean"], course_df["period_clean"]),
    course_df["id"]
))

person_lookup = dict(zip(
    person_df["full_name_clean"],
    person_df["UUID"]
))

```

Finally, the script generated **PersonTeachesCourse.csv** by assigning *courseId* and person *UUID* values, removing incomplete records and duplicates, and exporting the resulting many-to-many relationship table. Overall, the procedure transformed the raw LearnIT export into two database-ready CSV files.

An additional script was developed to create relationships between courses and pre-defined keywords by matching normalized keyword terms with tokenized course titles. The implementation generated a many-to-many relationship table, **CourseKeywords.csv**, linking *courseId* and *keywordId* values through exact word comparison. Although the approach was computationally efficient, it relied solely on direct lexical matches. Consequently, the keyword-based mapping was not incorporated into the final system design.

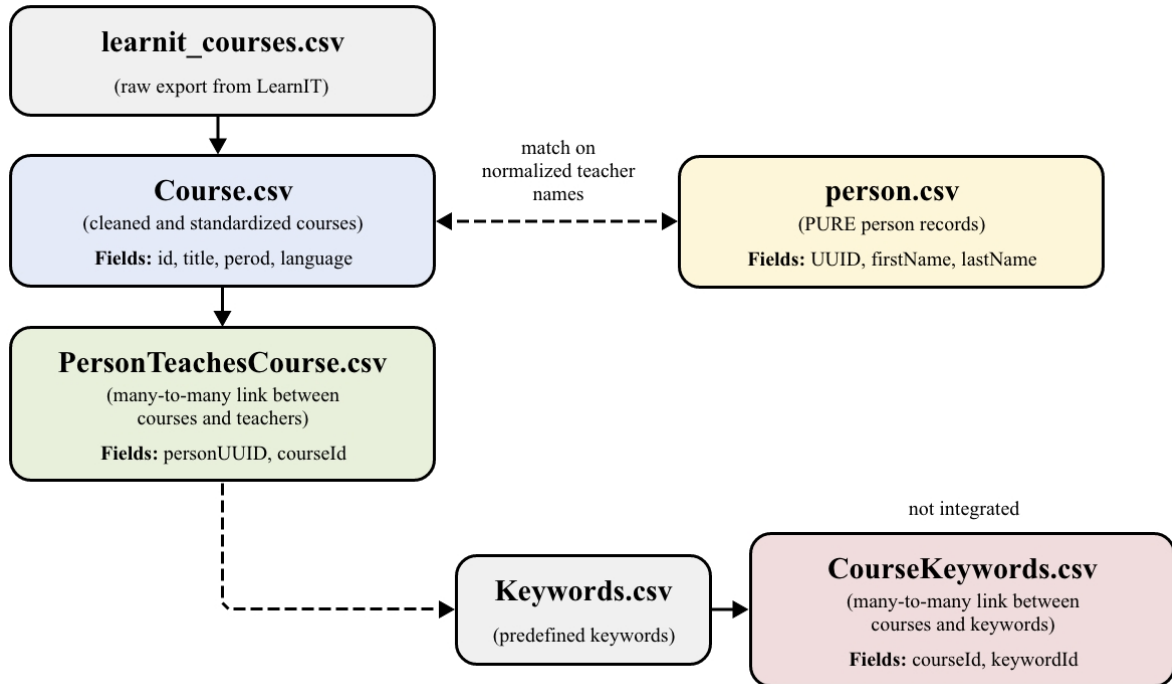


Figure 3: Relational structuring workflow for LearnIT course data

4 Architecture

As shown in Figure 4, the search engine is not deployed in a production environment and must be executed locally using Docker. The system was developed with a React and TypeScript-based front end, while the back end was implemented in Python using FastAPI and Whoosh for search functionality.

Both the front end and back end services are containerised independently and are deployed using Docker. The front end is accessible at `http://localhost:3000`, while the back end runs at `http://localhost:8000`. Internally, these services operate on ports 3000 and 8000 within Docker’s default virtual network bridge. These ports have also been exposed to the host machine to allow external access during local development.

The database is hosted externally on AWS RDS using PostgreSQL. The back end communicates with the database through a publicly accessible endpoint that permits connections from any IP address, provided valid credentials are supplied. While this configuration is suitable for development and testing purposes, it would not be appropriate for a production environment. In a production deployment, access would typically be restricted using fixed IP ranges or a secure virtual private network (VPN). However, such measures were not implemented in this project due to the absence of static IP addresses and the non-production nature of the system.

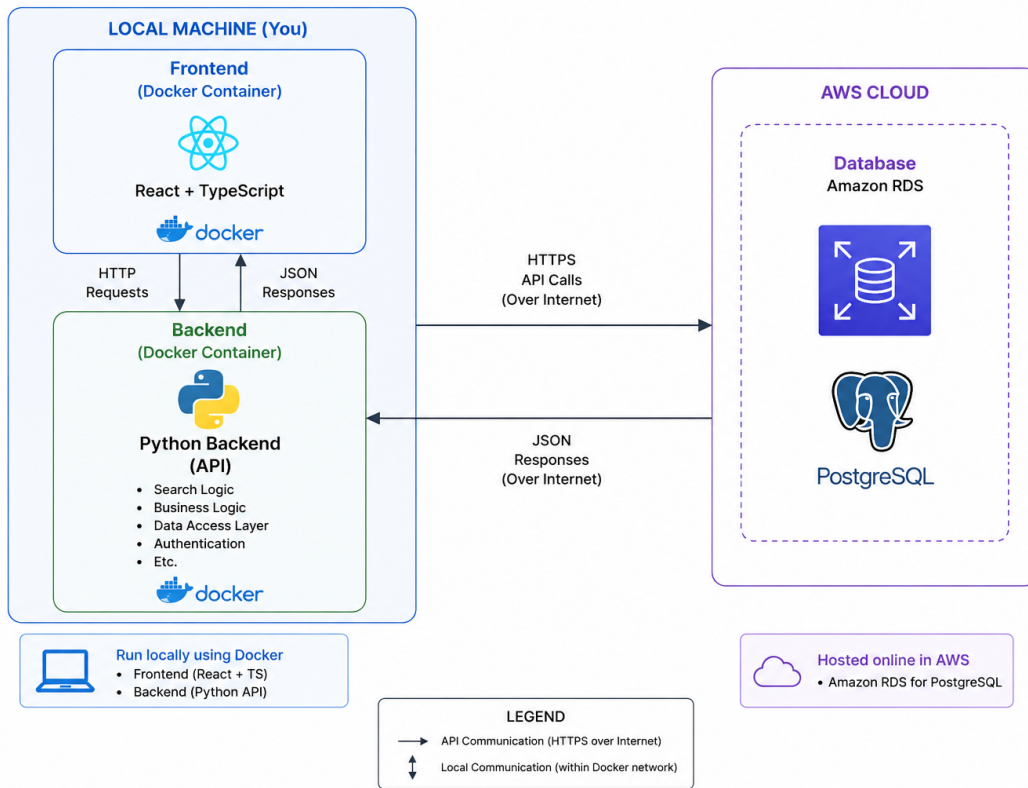


Figure 4: Expert Search Engine architecture and implementation

4.1 Database design

The database design was developed through an iterative feedback loop consisting of two complementary steps:

- **Requirement-driven design (working backwards):** The starting point of the design process was the identification of desired features for the search engine. This allowed us to infer the ideal structure and types of data that would be required from Pure, assuming full availability. This step defined what would be considered optimal.

- **Capability mapping:** In parallel, an analysis of Pure and the Danish Research Portal was conducted in order to determine which data fields were actually accessible through the available APIs. This included an examination of both object structures and API responses, as many fields were present in the schema but not consistently populated or used. This step defined the *feasible state* of the data model.

By iteratively combining these two perspectives, the data model was continuously refined. This process enabled the identification and definition of the required entities and their corresponding attributes (see subsequent subsection for full list of objects and fields). The resulting design was formalised into an Entity-Relationship (ER) model, shown in Figure 5.

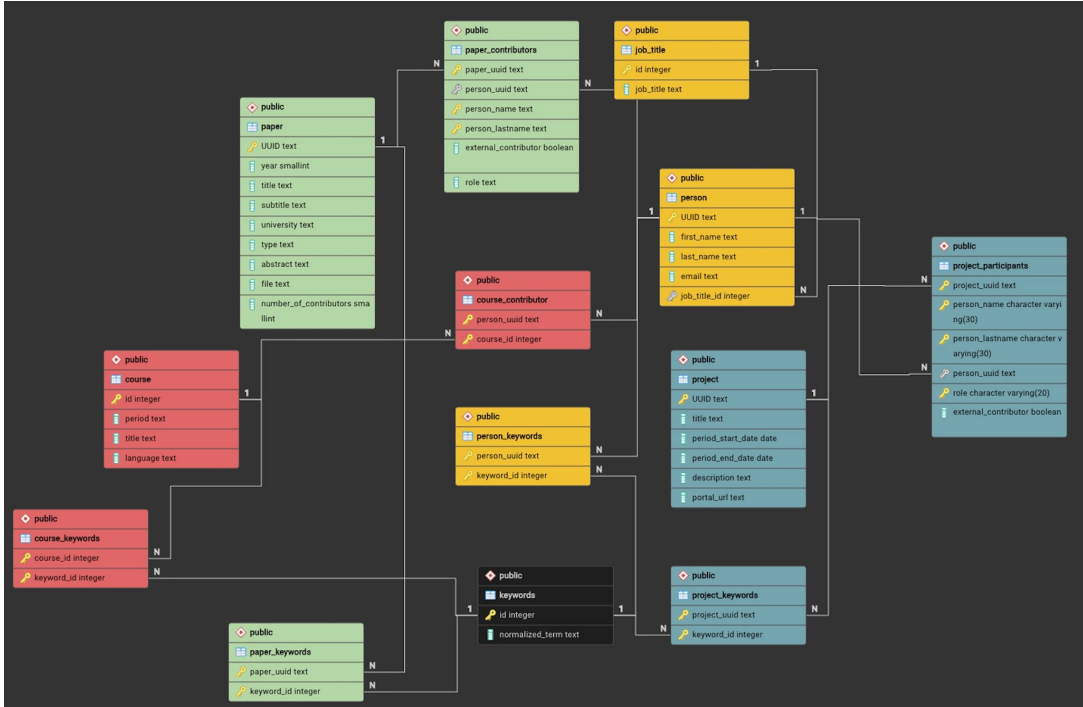


Figure 5: Entity-Relationship diagram of the final database schema

This approach is referred to as a feedback loop because the design continuously transitioned between three perspectives: the ideal representation of data (what would be optimal), the available representation (what exists in Pure), and the practical representation (what is actually populated and usable). In several cases, a field would initially appear desirable and would exist in the Pure schema; however, further analysis revealed that the field often contained sparse or inconsistent values, rendering it unsuitable for reliable use.

It is also important to note that, although the primary data source was Pure, the design was developed with the intention of integrating additional data sources, including the Danish Research Portal and LearnIT. This influenced the inclusion of certain entities, such as **Course**, which were designed to support multi-source integration.

The following subsection presents the final set of database entities and their corresponding attributes.

- **Person** The *Person* entity represents an individual in the system and is described by the following attributes:

- `uuid (string)`: Unique identifier.
 - `name (string)`: Full name.
 - `email (string)`: Contact email address.
 - `papers (string[])`: List of associated paper identifiers.
 - `courses (string[])`: List of associated course identifiers.
 - `projects (string[])`: List of associated project identifiers.
 - `job_title (string)`: Current job title/position.
 - `keywords (string[])`: List of keywords describing expertise/interests.
- Paper The *Paper* entity represents a research output and is described by the following attributes:
 - `uuid (string)`: Unique identifier.
 - `year (number)`: Publication year.
 - `title (string)`: Main title.
 - `subtitle (string)`: Subtitle (if available).
 - `university (string)`: Affiliated university.
 - `type (string)`: Publication type (e.g., article, conference paper).
 - `abstract (string)`: Abstract text.
 - `file_url (string)`: Link to the full text (if available).
 - `number_of_contributors (number)`: Number of contributors.
 - `contributors (string[])`: List of contributor identifiers.
 - `keywords (string[])`: Keywords describing the paper.
 - Course The *Course* entity represents a course and is described by the following attributes:
 - `id (number)`: Course identifier.
 - `period (string)`: Teaching period (e.g., semester/year).
 - `title (string)`: Course title.
 - `language (string)`: Language of instruction.
 - `contributors (string[])`: List of contributor identifiers (e.g., instructors).
 - `keywords (string[])`: Keywords describing course topics.
 - Project The *Project* entity represents a research project and is described by the following attributes:
 - `uuid (string)`: Unique identifier.
 - `title (string)`: Project title.
 - `period_start_date (date)`: Project start date.
 - `period_end_date (date)`: Project end date.
 - `description (string)`: Project description.
 - `participants (string[])`: List of participant identifiers.

4.2 Normalisation and Database creation

4.2.1 Normalization and indexing

During the database design phase, we targeted Third Normal Form (3NF) and, where feasible, Boyce–Codd Normal Form (BCNF) in order to follow established relational design best practices. In retrospect, this level of normalization was likely not strictly necessary for our use case: it introduced additional schema complexity (primarily through multiple junction tables) without providing a clear performance benefit, especially given the modest size of our dataset. Most tables contain fewer than 30,000 rows, with the main exception being the **Keywords** table which is currently not in use.

To mitigate potential performance bottlenecks in frequent queries, we also introduced indexes on commonly filtered and joined columns. Examples include an index on the **UUID** column in the **Person** table and an index on **person_uuid** in the **Course.Contributor** table.

4.2.2 Third Normal Form (3NF)

A relational schema is commonly considered to be in 3NF if the following conditions hold:

1. Every table has a primary key.
2. Every non-key attribute depends on the whole key.
3. Non-key attributes do not depend on other non-key attributes.

For a concise summary of normalization rules, see IBM’s documentation. [\[4\]](#)

4.2.3 Schema definition

The following SQL statements define the database schema used in this project.

Listing 8: Database schema (table creation statements)

```
CREATE TABLE Job_Title (
  id INTEGER PRIMARY KEY,
  Job_Title TEXT NOT NULL
);

CREATE TABLE Person (
  id UUID TEXT PRIMARY KEY,
  first_Name TEXT NOT NULL,
  last_Name TEXT NOT NULL,
  email TEXT,
  job_Title_Id INTEGER,
  FOREIGN KEY (job_Title_Id) REFERENCES Job_Title(id)
);

CREATE TABLE Course (
  id INTEGER PRIMARY KEY,
  period TEXT NOT NULL,
  title TEXT NOT NULL,
  language TEXT,
);
```

```

CREATE TABLE Keywords (
    id INTEGER PRIMARY KEY,
    normalized_term TEXT NOT NULL
);

CREATE TABLE Paper (
    UUID TEXT PRIMARY KEY,
    year smallint,
    title TEXT NOT NULL,
    subtitle TEXT,
    university TEXT,
    type TEXT,
    abstract TEXT,
    file TEXT,
    number_of_contributors smallint
);

CREATE TABLE Paper_Contributors (
    paper_UUID TEXT,
    person_UUID TEXT,
    person_Name TEXT NOT NULL,
    person_LastName TEXT NOT NULL,
    external_Contributor BOOLEAN NOT NULL,
    role TEXT,

    PRIMARY KEY (paper_UUID, person_Name, person_LastName),

    FOREIGN KEY (paper_UUID) REFERENCES Paper(UUID),
    FOREIGN KEY (person_UUID) REFERENCES Person(UUID)
);

CREATE TABLE Paper_Keywords (
    paper_UUID TEXT,
    keyword_Id INTEGER,

    PRIMARY KEY (paper_UUID, keyword_Id),

    FOREIGN KEY (paper_UUID) REFERENCES Paper(UUID),
    FOREIGN KEY (keyword_Id) REFERENCES Keywords(id)
);

CREATE TABLE Person_Keywords (
    person_UUID TEXT,
    keyword_Id INTEGER,

    PRIMARY KEY (person_UUID, keyword_Id),

    FOREIGN KEY (person_UUID) REFERENCES Person(UUID),
    FOREIGN KEY (keyword_Id) REFERENCES Keywords(id)
);

CREATE TABLE Course_Contributor (
    person_UUID TEXT,
    course_Id INTEGER,

    PRIMARY KEY (person_UUID, course_Id),

    FOREIGN KEY (person_UUID) REFERENCES Person(UUID),

```

```

        FOREIGN KEY(course_Id) REFERENCES Course(id)
    );

CREATE TABLE Project (
    UUID TEXT PRIMARY KEY,
    title TEXT NOT NULL,
    period_start_date DATE,
    period_end_date DATE,
    description TEXT,
    portal_URL TEXT
);

CREATE TABLE Project_Keywords (
    project_UUID TEXT,
    keyword_Id INTEGER,

    PRIMARY KEY (project_UUID, keyword_Id),

    FOREIGN KEY(project_UUID) REFERENCES Project(UUID),
    FOREIGN KEY(keyword_Id) REFERENCES Keywords(id)
);

CREATE TABLE Project_Participants (
    project_UUID TEXT,
    person_Name varchar(30),
    person_Lastname varchar(30),
    person_UUID TEXT,
    role varchar(20),
    external_Contributor BOOLEAN NOT NULL,

    PRIMARY KEY (project_UUID, person_Name, person_Lastname),

    FOREIGN KEY(project_UUID) REFERENCES Project(UUID),
    FOREIGN KEY(person_UUID) REFERENCES Person(UUID)
);

CREATE TABLE Course_Keywords (
    course_id INTEGER,
    keyword_Id INTEGER,

    PRIMARY KEY (course_id, keyword_Id),

    FOREIGN KEY(course_id) REFERENCES Course(id),
    FOREIGN KEY(keyword_Id) REFERENCES Keywords(id)
);

```

4.3 Search Engine - Back end

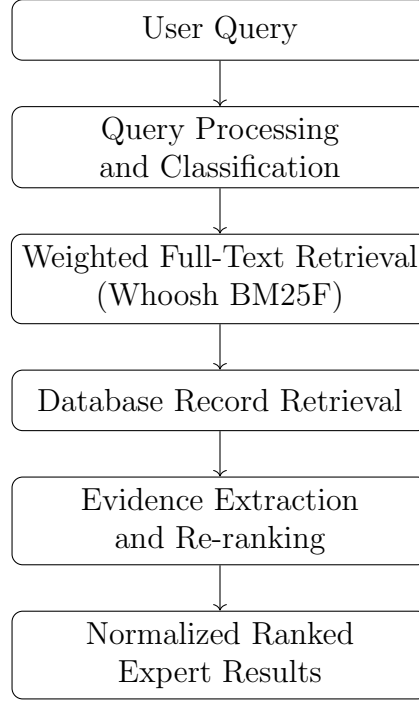
4.3.1 System Overview

The implemented system is an expert retrieval and ranking framework designed to identify and rank academic experts based on free-text user queries. The system combines lexical information retrieval techniques with post-processing relevance adjustments to improve the retrieval relevance, ranking precision, and interpretability of search results. Its primary objective is to enable efficient discovery of researchers and professionals by

leveraging structured expert profile data, including publication titles, publication abstracts, courses, projects, and project descriptions.

The architecture integrates two core technologies: the Whoosh full-text search engine for indexing and retrieval, and PostgreSQL for persistent storage of expert metadata. Queries submitted by users are processed through a multi-stage retrieval pipeline consisting of query normalization, query-type detection, weighted multi-field search, evidence extraction, score adjustment, and final result normalization.

The overall workflow of the system is illustrated conceptually as follows:



4.3.2 indexed_experts - flattened table

Our initial plan for implementing the search component was to rely on Elasticsearch. Consequently, we designed a dedicated, search-oriented table whose structure aligns with Elasticsearch’s recommended data modelling practices.

Elasticsearch documentation notes that, for many search workloads, *flattened (denormalized) document structures* are often preferable, as they can reduce the need for expensive nested queries and simplify retrieval.^[5]

Following this approach, we created a table that aggregates all content associated with a given expert into a single record. Specifically, each entry includes core person attributes (e.g., `id`, `name`, `email`, and `title`) alongside arrays of related textual fields such as `paper_titles`, `paper_abstracts`, `courses`, `projects`, and `project_descriptions`. The server queries this table whenever it receives a search request from the client, enabling a single lookup to retrieve the expert’s consolidated search representation.

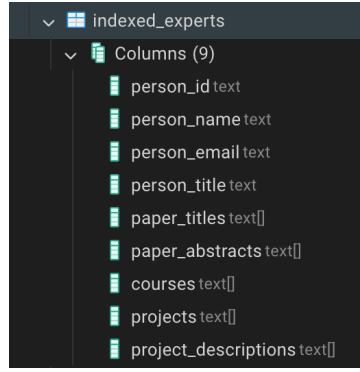


Figure 6: View of indexed_experts columns and data types in PostgreSQL

4.3.3 Data Normalization and Preprocessing

To ensure consistency during retrieval and ranking, all textual data underwent normalization before processing. The system converted all values into a standardized textual representation while handling missing or heterogeneous input formats. The `normalize_text()` function converted lists, strings, and null values into unified text representations:

```
def normalize_text(value:Any) -> str:
    if value is None:
        return ""
    if isinstance(value, list):
        return "␣".join(str(item) for item in value if item is not
                          None)
    return str(value)
```

This preprocessing step was necessary because expert metadata originated from multiple database fields with varying formats. Without normalization, inconsistencies in representation could negatively affect retrieval performance.

4.3.4 Query Classification

A central component of the system was the classification of user queries into semantic categories. The implemented approach distinguishes between:

1. Single-token queries
2. Full-name queries
3. Mixed topical queries

The classification process was implemented in the `detect_query_type()` function:

```
def detect_query_type(query_text : str) -> str:
    words = query_text.strip().split()
    if len(words)==1:
        return "single_token"
    if len(words)!=2:
        return "mixed"
    q_norm = normalize_name(query_text)
    for hit in name_probe(query_text, limit=5):
        if normalize_name(hit["person_name"]) == q_norm:
            return "full_name"
    return "mixed"
```

This distinction was important because different query types required different ranking strategies. For example, a query consisting of a person's full name should prioritize exact

identity matching, whereas topical queries should prioritize semantic relevance within publications and projects. For example:

Query	Detected Type	Intended Meaning
"John Smith"	full_name	Search for a specific person
"robotics"	single_token	Topic or surname
"deep learning healthcare"	mixed	Research topic search

Table 1: Examples of query type detection and interpretation

The classification stage, therefore enabled adaptive retrieval behavior depending on the user’s intent.

4.3.5 Multi-Field Retrieval Using BM25F

The retrieval stage used the Whoosh search engine with the BM25F ranking algorithm. BM25F is an extension of the traditional BM25 probabilistic retrieval model that supports weighted scoring across multiple document fields [6].

The search index contained the following searchable fields:

- Person name
- Paper titles
- Paper abstracts
- Courses
- Projects
- Project descriptions

Field-specific importance was controlled through manually defined boost values:

```
return {
    "person_name": person_name_boost,
    "paper_titles": 3.0,
    "projects": 2.4,
    "project_descriptions": 2.0,
    "paper_abstracts": 1.6,
    "courses": 1.1,
}
```

Publication titles received the highest weighting because they generally contain concise and highly informative representations of research expertise[7]. Projects and project descriptions were also strongly weighted due to their relevance in representing applied expertise and ongoing work [8] Course information received lower weighting because teaching topics are often broader and less discriminative than research outputs.

The retrieval process was implemented using a **MultifieldParser**:

```
parser=MultifieldParser(
    [
        "person_name",
        "paper_titles",
```

```

    "projects",
    "project_descriptions",
    "paper_abstracts",
    "courses",
    ],
    schema=ix.schema,
    fieldboosts=boosts,
    group=OrGroup.factory(0.9),
)

```

The use of an OR-based query grouping strategy improved recall by allowing partial term matches rather than requiring exact matching of all query terms.

4.3.6 Candidate Retrieval and Database Integration

The search engine returns candidate identifiers together with relevance scores. These identifiers are subsequently used to retrieve complete expert profiles from the PostgreSQL database.

The retrieval query is executed as follows:

```

SELECT
person_id,
person_name,
person_email,
paper_titles,
paper_abstracts,
courses,
projects,
project_descriptions
FROM indexed_experts
WHERE person_id=ANY(%s)

```

This separation between the search index and relational database provided two advantages. First, it improved retrieval efficiency by storing only optimized searchable content. Second, it preserved database flexibility by allowing the storage of richer structured meta-data outside the index itself.

4.3.7 Score Adjustment and Re-ranking

The initial BM25F retrieval score was further refined using heuristic ranking adjustments. The system incorporates query-aware name matching bonuses and evidence-based boosts.

The score adjustment process was defined as:

```

adjusted_score=base_score+name_bonus
if is_person_query and name_bonus > 0:
    adjusted_score+=5.0
if not is_person_query and has_relevant_evidence(evidence):
    adjusted_score+=2.0

```

Exact full-name matches received the largest ranking bonuses:

```

if q==name:
    return 40.0 if query_type=="full_name" else 20.0

```

This strategy ensured that person-oriented searches strongly prioritize identity matching while topical searches prioritize semantic evidence.

Additionally, topical searches excluded experts lacking relevant supporting evidence:

```
if not is_person_query and not has_relevant_evidence(evidence):  
    continue
```

This filtering step reduced false positives and improved result precision.

4.3.8 Score Normalization

Because raw BM25F scores vary depending on query composition and corpus statistics, the final ranking scores are normalized to a range between 0 and 1.

The normalization process was implemented as:

```
(result["score"]-min_score)/(max_score-min_score)
```

This normalization improved interpretability and enabled easier integration with frontend interfaces and visual ranking systems.

4.3.9 REST API with Fast-API

To enable communication between the front-end application and the server-side functionality, we implemented a REST API using the FastAPI framework. The endpoints are defined in `App/server/api.py` and provide the functionality required by the client to display both search results and detailed expert pages.

- `/search?q`

Purpose: Executes a search query and returns a ranked list of matching experts. The response includes the query metadata (e.g., query string, query type, and total count) as well as an array of experts. For each expert, the API returns the UUID and basic identifying information (name, surname, email), together with up to ten related items in each of the following categories: papers, projects, and courses. This endpoint is used by the front end to render the search results page.

Returns (schema):

```
query: string;  
query_type: string;  
count: number;  
results: Expert[];
```

Example request:

```
GET http://127.0.0.1:8000/search?q=marco%20carbone&limit=15 HTTP  
/1.1
```

- `/person?person_uuid`

Purpose: Retrieves the core profile data for a specific expert. The response contains the expert UUID and a list of person-related attributes, including first name, last name, email, and job title. This endpoint is used by the front end to render the expert detail page.

Returns (schema):

```
person_uuid: string;  
data: PersonData[];
```

Example request:

```
GET http://127.0.0.1:8000/person?person_uuid=9027b41a-4159-4db7-9
ddd-8fc8f41f1fb3 HTTP/1.1
```

- `/papersByPerson?person_uuid`

Purpose: Retrieves all papers associated with a given expert. Each paper includes fields such as title, subtitle, year, abstract, and `file_url`. This endpoint is used by the front end to populate the papers section on the expert detail page.

Returns (schema):

```
person_uuid: string;
papers: Paper[];
```

Example request:

```
GET http://127.0.0.1:8000/papersByPerson?person_uuid=9027b41a
-4159-4db7-9ddd-8fc8f41f1fb3 HTTP/1.1
```

- `/projectsByPerson?person_uuid`

Purpose: Retrieves all projects associated with a given expert. Each project includes the title, start and end dates, and (when available) a description. This endpoint is used by the front end to populate the projects section on the expert detail page.

Returns (schema):

```
person_uuid: string;
projects: Project[];
```

Example request:

```
GET http://127.0.0.1:8000/projectsByPerson?person_uuid=9027b41a
-4159-4db7-9ddd-8fc8f41f1fb3 HTTP/1.1
```

- `/coursesByPerson?person_uuid`

Purpose: Retrieves all courses associated with a given expert. Each course includes the course title and teaching period. This endpoint is used by the front end to populate the courses section on the expert detail page.

Returns (schema):

```
person_uuid: string;
courses: Course[];
```

Example request:

```
GET http://127.0.0.1:8000/coursesByPerson?person_uuid=9027b41a
-4159-4db7-9ddd-8fc8f41f1fb3 HTTP/1.1
```

4.4 Web application - Front End

Our front end is implemented as a Single-Page Application (SPA) using TypeScript and React. Since the objective was to deliver a Minimum Viable Product (MVP), the first iteration focused on the following design goals:

- **Simplicity and noise reduction:** the interface includes only the elements strictly necessary for completing the primary user tasks.
- **Context and transparency:** to avoid replicating the limited transparency observed in existing systems (e.g., Pure/DRP) regarding how results are returned and ranked, the interface provides explanatory context on the ranking and the factors contributing to the output.
- **Overview vs. detail:** the application supports both a high-level overview of multiple experts matching a query and a detailed view for inspecting the full set of information available for a selected expert.

The application renders three pages, organized in the following structure:

```
App
-> SearchPage
  -> SearchResultsPage
    -> ExpertDetailPage
```

Communication with the server is handled via the built-in `fetch()` function in TypeScript/JavaScript. The client-side API calls are implemented in `App/client/src/api`, where dedicated functions connect to the corresponding server endpoints and are then consumed by page-level hooks.

The three pages are summarized below:

- **Search page:** provides a short explanation of how the search algorithm works and offers guidance for achieving better results, particularly for multi-word queries. The page forwards the query parameter `q` (as a URI component) to the Search Results page. See Figure 7.

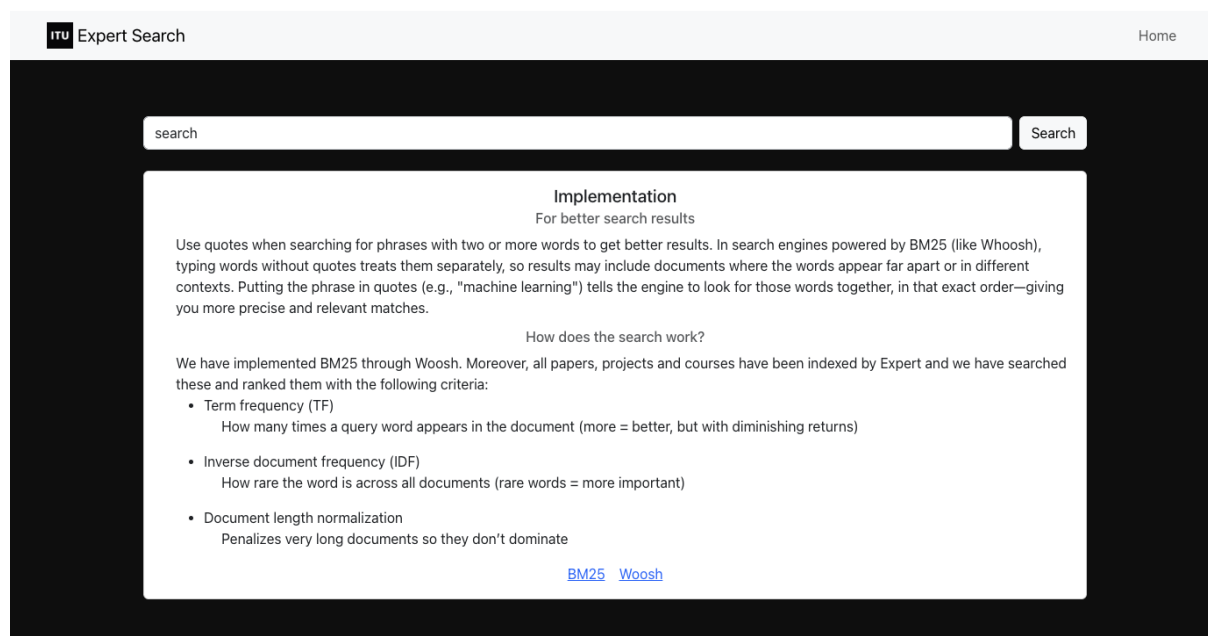


Figure 7: View of Search page in browser

- **Search results page:** displays cards for up to 15 experts whose papers, projects, or courses match the query. Using the query `q`, the page calls the server endpoint

`/search?q` and retrieves a ranked expert list, including up to 10 representative items per expert for each category (**paper**, **project**, and **course**) to provide a concise overview of the expert's main activities. When the user selects *See more*, the expert's `person_uuid` is passed (as a URI component) to the Expert Detail page. See Figure 8.

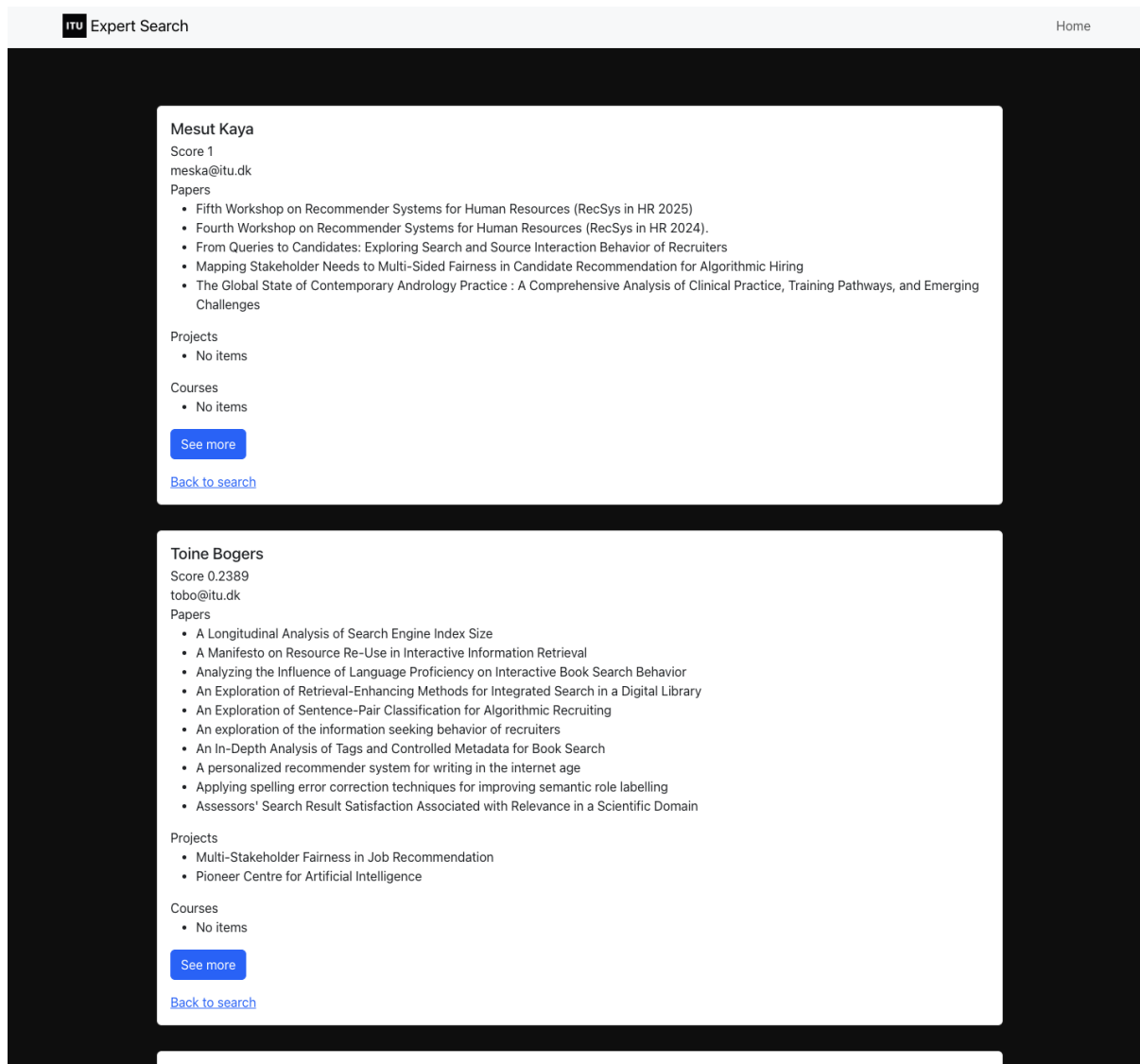


Figure 8: View of Search results page in browser

- **Expert detail page:** presents the complete set of available information for a selected expert. Using the provided `person_uuid`, the page queries `/person?person_uuid`, `/coursesByPerson?person_uuid`, `/projectsByPerson?person_uuid`, and `/papersByPerson?person_uuid`. These endpoints return the expert's profile information (e.g., name, title, and email) together with the full lists of associated papers (as well as a link to each publication if this was available), projects, and courses. See Figure 9.

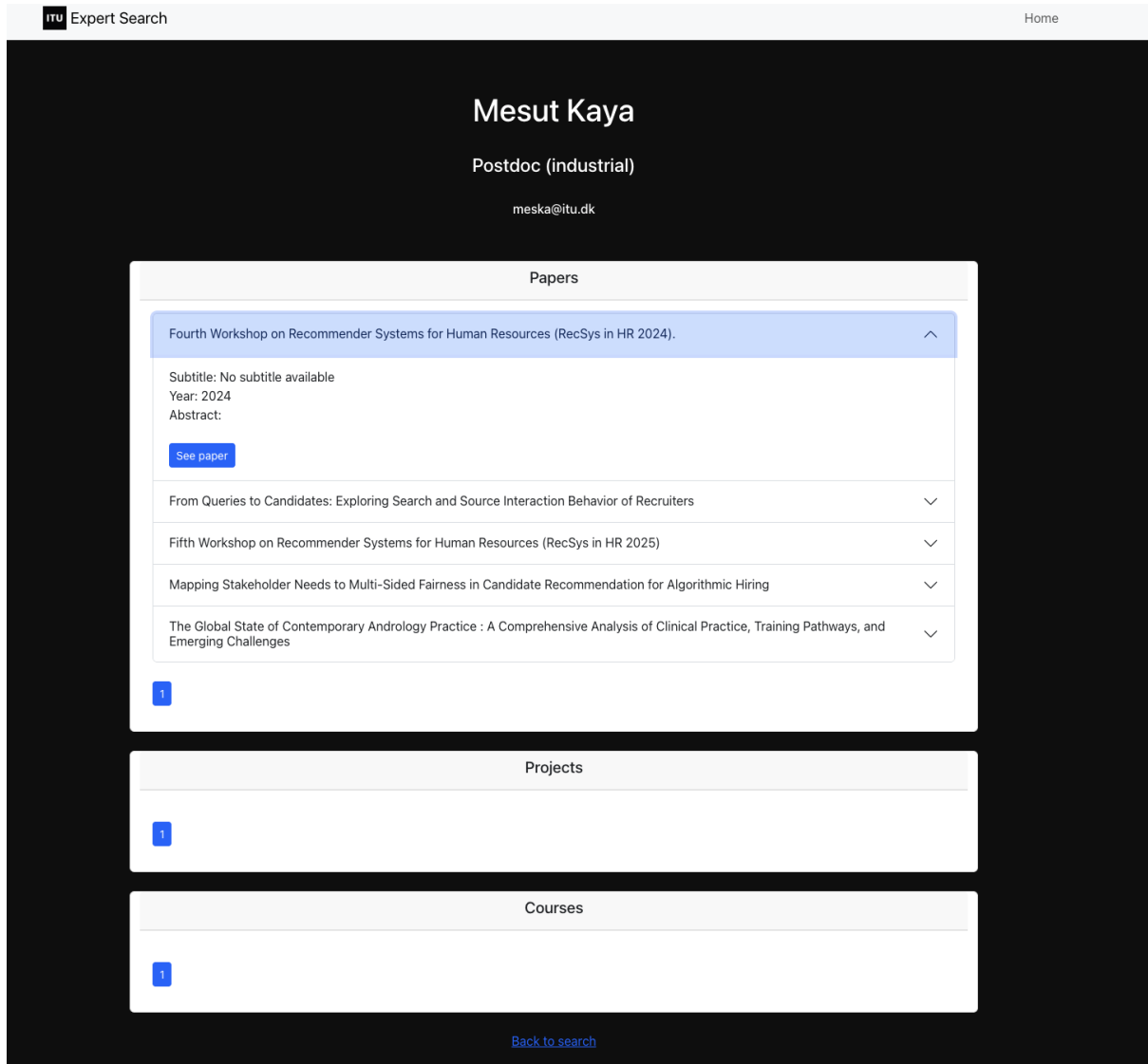


Figure 9: View of Expert detail page in browser

5 Discussion and Conclusion

5.1 Limitations

5.1.1 Data collection

Unfortunately, there are falsely matched paper contributions in our dataset, which is a major flaw.

During the data expansion phase, we aimed to enrich ITU researcher profiles by linking **Person** records extracted from Pure to their corresponding publications in the DRP. However, although ORCID identifiers were available in the Pure export, we did not incorporate them into the matching procedure. Instead, publications were linked to authors solely based on name matching. This approach introduced systematic errors for individuals with common Danish names, as multiple distinct authors in the DRP may share identical or highly similar names.

As a result, the dataset contains approximately 20 persons who appear to have more

than 200 publications each. Given the prevalence of their names, these counts are implausible and strongly indicate erroneous author–publication associations. This issue was identified only around one week before the submission deadline, and we were therefore unable to address it in time. Correcting the problem would require redesigning parts of the database schema (to incorporate reliable author identifiers such as ORCID) and rerunning the full ETL process.

This constitutes a major limitation of the current system and reduces the reliability of the search results. The database contains approximately 32,000 papers in total; if the 20 most “productive” experts are (incorrectly) associated with roughly 20,000 of these papers, the ranking and relevance signals used by the search engine are substantially distorted, even if there is overlap among the misassigned publications.

Figure 10 presents the distribution of paper counts per person in the current database.

	papers_count integer	person_name text	person_uuid text
1	3590	Søren Nielsen	545501c5-18f2-4cf9-84da-3348bbef2...
2	2892	Jesper Nielsen	82e44c10-e721-4308-8c18-cbe0f5ece...
3	2071	Feng Feng	dac0e506-da5d-4ee6-8783-5abd2a20...
4	2038	Henrik Larsen	07a52bc0-1368-4c96-b20d-4bf9668a3...
5	1424	Anders Rasmussen	9228d512-99fd-4254-a4c9-f782687d2...
6	1136	Charlotte Larsen	d49a30e4-939a-4a5c-98bf-adc795e89...
7	1041	Lene Nielsen	1052b289-f510-4c5a-9662-2db48c12e...
8	738	Louise Sørensen	193e7067-841e-4b93-8713-a3df2bc2c...
9	684	Søren Knudsen	2575c800-642d-4984-9c56-41601cf07...
10	569	René Hansen	14396cd2-ae59-4ac5-8569-e0305c3a...
11	515	Jens Schmidt	e0edf6ca-5a14-41d8-81cc-77fd67f03e...
12	513	Xiao Li	cfd2eb14-8c3d-4c92-b6f4-acf076e821...
13	439	Line Larsen	640cf04f-70c5-419a-abea-69d1f0330...
14	375	Carsten Jørgensen	3fc4cf72-04f9-43a3-91ff-fc629a0bbb46
15	348	Matthias Nielsen	19286916-2423-4158-99db-da82d97b...
16	295	Kristina Andersen	6bc60ad3-a8f5-4f7b-9f76-ee5cec4e2...
17	278	Tobias Pedersen	cac3823d-3dc6-4513-8448-be21a2d4...
18	262	Wei Guo	bc8bcc0c-c956-451c-8f18-6caed523f...
19	258	Nicholas Hansen	5faaedca-53c9-4888-b617-2d81414c...
20	224	Rikke Meyer	33e8ee22-baab-489b-8fb2-7d931d7f1...
21	224	Allan Petersen	18d9e93f-cab1-48d8-be6e-a0b9aa08b...
22	209	Jacob Møller Jensen	0f625ea4-4ff9-4feb-8f1c-7ff3abfc0c0b
23	200	Jan Pries-Heje	79ebe3ed-0527-4160-9af6-be0b0853b...
24	172	Thomas Hildebrandt	d30b96a8-4f1e-44fa-9bf5-0209bedf02...
25	163	Andrzej Wasowski	a39b94ff-e1b6-4b96-8404-486167a91...
26	161	Thomas Binder	c90e23c6-5bf4-4d6f-97d8-b162b8a5b...
27	144	Sebastian Risi	a420ffb3-2ca6-4925-a25f-baaa5228fc...
28	135	Annette Thomsen	15d90e32-9373-4257-9a74-2b92124fc...
29	134	Toine Bogers	e726f512-d3fd-4d89-962a-3aa9f532a...

Figure 10: View of Expert detail page in browser

5.1.2 Search engine

Despite its effectiveness, the current implementation has several limitations.

First, the system relies primarily on lexical matching and therefore cannot capture deeper semantic relationships between terms. Queries using synonyms or semantically related terminology may therefore produce suboptimal retrieval results.

Second, the manually defined field boosts and heuristic scoring bonuses require empirical tuning and may not generalize optimally across all domains.

Additionally, the current implementation exposes a fallback database password directly in the source code. In production environments, credentials should instead be managed securely through environment variables or dedicated secret-management systems.

5.2 Improvements

5.2.1 Search Engine - Back end

Future improvements could include:

- **Semantic and Hybrid Retrieval** Integration of semantic embeddings (using FAISS) and/or hybrid lexical-semantic retrieval.
- **Configurable Search Algorithms** Development of several search algorithms and the possibility to switch among them.
- **Publication Weighting** Introduction of additional (positive) weights on publications based on the year of publication.
- **Keyword-Based Search** Implementation of keyword search.
- **Academic Profile Management** Allowing academics to log into the portal through email verification, so that they could provide; a summary about themselves, a picture, their main topics of interest, availability to calendar, location in Campus(office).
- **AI-Generated Expert Matching Summaries** Development of an AI model which takes person UUID together with retrieved papers, projects, and courses as input, and generates a concise 2–3 line explanation of why a particular expert is a strong match for the search

5.2.2 Data collection

- **ORCID-based matching:** This is expected to provide the largest overall improvement in data quality and, consequently, in search performance. We would redesign the `Person` and `Paper_Contributors` tables to include the ORCID field available in the Pure export. We would then rerun the data collection and ETL scripts, matching DRP authors to publications exclusively via ORCID rather than name-based heuristics. While this would likely reduce the total number of extracted contributions from the DRP, it would substantially improve precision, eliminate many false matches, and still allow us to expand the dataset beyond what is available in Pure alone.
- **ETL pipeline:** We could implement an automatic - periodic extraction, transformation, loading process that would run once or twice yearly, to keep data up to date.
- **Keywords:** Revise how we collected and inserted keywords on the database - however, this will greatly depend on how we decide to use them.

5.2.3 Front-end

- **Expert Detail page:**

- Add an on-page search field to refine and display only content related to a new query within the selected expert’s context.
- Improve pagination by implementing dynamic pagination (e.g., a sliding-window approach).
- Enrich expert profiles with additional attributes, such as availability, topics of interest, an AI-generated summary, a self-authored summary, and office location.
- Hide project and course sections when no corresponding results exist (to reduce visual noise).

- **Search Results page:**

- Return only results that are relevant to the query and highlight matched query terms in the displayed results.

5.2.4 Deployment

Moreover, the tool could be deployed for open access. A polished version of the system could be made publicly accessible. For a lightweight second iteration, this could be achieved through free-tier hosting. If future versions require more resources or long-term maintenance, hosting through ITU would be preferable.

5.3 Conclusion

The developed expert search framework demonstrates how traditional information retrieval techniques can be adapted for expert discovery applications. By combining BM25F-based multi-field retrieval with query-type-aware ranking adjustments and evidence extraction, the system provides relevant, interpretable, and context-sensitive expert recommendations.

The architecture offers a scalable and modular foundation for academic expert search systems and can be extended in future work through semantic retrieval techniques and machine learning-based ranking approaches.

The implemented system provided several advantages:

- Efficient multi-field retrieval using BM25F
- Query-aware ranking adaptation
- Separation of indexing and persistent storage
- Flexible support for both person-oriented and topic-oriented queries

The combination of lexical retrieval and heuristic re-ranking enabled the system to balance precision, recall, and interpretability in expert recommendation scenarios. That being said, the data collection flaw that we mentioned in the subsection Limitations greatly impacted the quality of search results and made it impossible for us to scientifically assess the quality of our implementation.

Therefore, a direction for future work in this project should be the systematic evaluation of retrieval quality. Beyond the improvements proposed throughout this research, subsequent efforts should prioritize assessing whether the implemented changes lead to measurable enhancements in the relevance of retrieved results. This can be achieved by issuing a representative set of queries to the search engine and conducting a structured analysis of the resulting rankings. The retrieved experts should then be compared against an expected relevance baseline derived from the project objectives and domain expertise, enabling a clearer assessment of how well the system aligns with the intended expert-finding behavior. Although such end-to-end retrieval quality validation was outside the primary scope of the present study, it represents a critical next step for demonstrating the practical effectiveness and real-world impact of the proposed approach.

6 Use of AI

In accordance with the guidelines published on the ITU student website [9], we used generative AI (GAI) tools in a limited and supporting role throughout the project:

- as a personal teaching assistant and sparring partner when discussing architectural patterns;
- during early-stage ideation, including brainstorming and comparing alternative implementations of the database, search engine, and API design;
- for language support, including grammar correction and summarisation. All content was authored by the team; drafts were proofread using GitHub Copilot and ChatGPT, and subsequently revised by us;
- as a coding advisor to validate our understanding while writing the report in $\text{\LaTeX}$, developing the front end in React and TypeScript, implementing the search engine in Python, and developing data collection scripts in TypeScript/JavaScript and Python.

References

- [1] Juri Rügger. Supervisor–student matching with semantic embeddings. Master’s thesis, IT University of Copenhagen, 2025.
- [2] Alexandra-Afrodita Ghizdareanu and Filippo Piloni. Designing transparent thesis-matching platforms: A user-centered approach to better research outcomes. Master’s thesis, IT University of Copenhagen, 2026.
- [3] Edith Cowan University. What does doi mean and how do i use a doi in citations/referencing?, 2025. URL: <https://askus2.ecu.edu.au/s/article/000003401>.
- [4] IBM. Summary of normalization rules, 2026. URL: <https://www.ibm.com/docs/en/informix-servers/15.0.x?topic=model-summary-normalization-rules>.
- [5] ElasticSearch. Nested field type, 2026. URL: <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/nested>.
- [6] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi:10.1561/15000000019.
- [7] University of Eastern Finland. Search fields, 2025. URL: <https://sites.uef.fi/tiedonhaku-kirjasto/search-fields/?lang=en>.
- [8] Krisztian Balog, Yi Fang, Maarten de Rijke, Pavel Serdyukov, and Luo Si. Expertise retrieval. *Foundations and Trends in Information Retrieval*, 6(2–3):127–256, 2012. doi:10.1561/15000000024.
- [9] ITU. What can i use generative ai for?, 2026. URL: <https://itustudent.itu.dk/Study-Administration/Generative-AI/What-can-I-use-generative-AI-for>.